

前端面试题库

由 37 份前端面试资料整理蒸馏而成

目录

1. JavaScript 基础
 2. JavaScript 进阶
 3. ES6+
 4. 异步编程 (Promise/async/await)
 5. HTML
 6. CSS
 7. 浏览器原理
 8. 网络协议
 9. 性能优化
 10. Vue
 11. React
 12. 工程化 (Webpack/模块化)
 13. TypeScript 与 Axios
 14. 安全
 15. 手写代码
 16. 大厂面试真题
-

1. JavaScript 基础

Q: JavaScript 中的基本类型有哪些？各自如何存储？

答案：原始类型：String、Number、Boolean、null、undefined、Symbol、BigInt。引用类型：Object、Array、Date、RegExp、Function。原始类型按值存放在栈中，引用类型在堆内存中，栈中保存引用地址。

Q: 0.1 + 0.2 为什么不等于 0.3？

答案：JS 采用双精度浮点数（64位），0.1 的二进制是无限循环小数，浮点数标准截取导致精度丢失。0.1 变为 0.100...001，0.2 变为 0.200...002，所以相加大于 0.3。可通过 `Number.EPSILON` 判断浮点数相等。

Q: `typeof null` 为什么是 'object'？

答案：JS 最初版本使用 32 位系统，用低位存储变量类型信息，000 开头代表对象，null 表示为全零，所以被错误判断为 object。

Q: 如何正确获取变量的类型？

答案：`Object.prototype.toString.call(x)` 返回 `[object Type]` 格式字符串，可准确判断所有内置类型。

Q: 判断数组的方法有哪些？

答案： 1. `Array.isArray()` (ES6 推荐)；2. `Object.prototype.toString.call(arr) === '[object Array]'`；3. `arr instanceof Array` (跨 iframe 可能失效)；4. `arr.constructor === Array`。

Q: `==` 和 `===` 的区别？

答案： `===` 严格相等，比较类型和值。`==` 允许类型转换后比较。规则：字符串与数字比较时字符串转数字；布尔与其他比较时布尔转数字；对象与基本类型比较时对象调用 `valueOf()`/`toString()` 转换。

Q: 哪些值强制转布尔时为 `false`？

答案： `undefined`、`null`、`false`、`NaN`、`''`、`0`、`-0`、`0n` (BigInt)。其余全为 `truthy`。

Q: `var`、`let`、`const` 的区别？

答案： 1. 作用域：`var` 函数/全局作用域，`let/const` 块级作用域；2. 变量提升：`var` 提升为 `undefined`，`let/const` 存在暂时性死区 (TDZ)；3. 重复声明：`var` 可重复，`let/const` 不可；4. `const` 必须初始化，且保证内存地址不变 (引用类型可改属性)。

Q: 什么是暂时性死区 (TDZ)？

答案： `let` 和 `const` 声明的变量在声明之前不能被访问，这个区域称为暂时性死区。

Q: 什么是闭包？

答案： 闭包是指有权访问另一个函数作用域中变量的函数。特点：1. 是一个函数；2. 能访问上级函数作用域中的变量 (即使上级函数已执行完)。应用场景：模块化、函数柯里化、防抖节流、循环中保存变量。

Q: 闭包的应用场景有哪些？

答案： `setTimeout` 传参、回调函数、IIFE、函数防抖 (debounce) 和节流 (throttle)、柯里化 (currying)、模块化模式。

Q: 闭包的经典面试题：`for` 循环中 `var` 和 `let` 的区别？

答案： 假设 `for (var i = 0; i < 3; i++) { setTimeout(() => console.log(i)) }` 输出全是 3，`var` 是函数/全局作用域，循环结束后 `i=3`，所有定时器回调共享同一个 `i`。`let i` 输出 0 1 2，每次循环创建新变量，闭包引用的是块级作用域中的 `i`。用 `var` 可通过 IIFE 创建闭包解决。

Q: `this` 的绑定规则有哪些？

答案： 1. 默认绑定 (非严格模式 $\rightarrow$ window，严格模式 $\rightarrow$ undefined)；2. 隐式绑定 (`obj.fn()` $\rightarrow$ `obj`)；3. 显式绑定 (`call`/`apply`/`bind`)；4. `new` 绑定 (新对象)；5. 箭头函数 (外部作用域 `this`)。

Q: 箭头函数和普通函数的区别？

答案： 箭头函数没有自己的 `this` (从外部作用域继承)，不能通过 `bind`/`call`/`apply` 修改 `this`，不能作为构造函数 (不能 `new`)，没有 `arguments` 对象 (可用 `rest` 参数代替)，没有 `prototype`。

Q: `call`、`apply`、`bind` 的区别？

答案： call 接受参数列表，apply 接受参数数组，两者都立即执行。bind 返回绑定 this 的新函数（不立即执行），支持柯里化。

Q: new 操作符执行时发生了什么？

答案： 1. 创建空对象；2. 该对象的 `__proto__` 指向构造函数的 prototype；3. 执行构造函数，this 绑定到新对象；4. 如果构造函数返回对象则返回该对象，否则返回新创建的对象。

Q: 什么是作用域链？

答案： 当访问变量时，解释器在当前作用域查找，未找到则去父作用域找，直到全局作用域。这种由内向外逐层查找的链式结构称为作用域链。

Q: 变量提升与函数提升的规则？

答案： 函数提升优先于变量提升。函数声明整体提升，var 变量只提升声明（赋值为 undefined）。let/const 不提升。

Q: JavaScript 的执行上下文有哪些？

答案： 三种：全局执行上下文、函数执行上下文、eval 执行上下文。每个上下文包含：变量对象（VO）、作用域链、this。

Q: 什么是执行栈（调用栈）？

答案： 具有 LIFO 结构，用于存储执行上下文。栈底是全局执行上下文，栈顶是正在执行的函数上下文。函数执行完即出栈。

Q: 基本类型和引用类型在内存中如何存放？

答案： 基本类型保存在栈内存中，按值访问。引用类型保存在堆内存中，栈中存放访问地址，按引用访问。闭包中的变量保存在堆中。

Q: 赋值、浅拷贝、深拷贝的区别？

答案： 赋值：基本类型互不影响，引用类型指向同一对象。浅拷贝：第一层基本类型独立，引用类型仍共享地址。深拷贝：所有属性完全拷贝，互不影响。

Q: 如何实现深拷贝？

答案： 使用递归 + WeakMap 解决循环引用。用 `Reflect.ownKeys()` 获取所有键（含 Symbol）。遍历属性，值为对象时递归，基本类型直接拷贝。

Q: JSON.parse(JSON.stringify(obj)) 实现深拷贝有什么缺陷？

答案： 1. 忽略 undefined；2. 忽略 Symbol；3. 不能序列化函数；4. 循环引用会报错；5. Date 会被转成字符串（反序列化后不是 Date 对象）；6. 正则/Error 等特殊对象会被转成空对象 `{}`。

Q: 原型和原型链是什么？

答案：每个函数都有 prototype（显式原型），每个实例都有 `__proto__`（隐式原型），实例的 `__proto__` 指向构造函数的 prototype。原型链从对象的 `__proto__` 开始逐层连接，用于实现继承。

Q: JavaScript 如何实现继承？

答案：6 种方式：1. 原型链继承；2. 构造函数继承（call/apply）；3. 组合继承；4. 寄生组合继承（最佳方式）；5. 原型式继承；6. ES6 class extends。

Q: ES5 和 ES6 继承的区别？

答案：ES5 先创建子类实例，再将父类方法添加到 this 上。ES6 先创建父类实例（必须先调 super()），再修改 this。class 声明内部启用严格模式，且不会提升。

Q: typeof 能判断哪些类型？

答案：基本类型（除 null）返回正确类型。引用类型除 function 外都返回 'object'。typeof null 返回 'object'（历史 bug）。

Q: instanceof 的原理？

答案：判断构造函数的 prototype 是否在实例的原型链上。实现：循环获取实例的 `__proto__`，与构造函数的 prototype 比较。

Q: parseInt 的用法和规则？

答案：`parseInt(string, radix)`，radix 是 2-36 的基数。遇到不属于指定基数的字符则停止解析。如 `parseInt('52px')` 返回 52。

Q: Set、Map、WeakSet、WeakMap 的区别？

答案：Set 存储唯一值，可遍历。WeakSet 成员只能是对象，弱引用，不可遍历。Map 键值对集合，键可是任意类型，可遍历。WeakMap 键只能是对象，弱引用，不可遍历。

Q: 数组去重的方法？

答案：1. ES6 Set: `[...new Set(arr)]`；2. 利用空对象做哈希表；3. filter + indexOf。

Q: 数组的原生方法分类？

答案：改变自身：pop/push/shift/unshift/splice/sort/reverse/fill/copyWithin。不改变：concat/slice/join/indexOf/includes/toString。遍历：forEach/map/filter/reduce/some/every/find/findIndex/flat。

Q: 类数组如何转为数组？

答案：1. ES6 `Array.from()`；2. `Array.prototype.slice.call()`；3. 扩展运算符 `[...args]`（可迭代对象）。

Q: 什么是事件代理（事件委托）？

答案：将事件绑定到父元素上，利用事件冒泡机制，让父元素代为处理子元素的事件。优点：减少事件注册数量，节省内存，动态添加的子元素自动拥有事件处理。

Q: 事件传播的三个阶段是什么？

答案：捕获阶段（window → target）→ 目标阶段 → 冒泡阶段（target → window）。`addEventListener` 第三个参数为 `true` 时在捕获阶段触发，为 `false`（默认）时在冒泡阶段触发。

Q: 如何阻止事件冒泡和默认行为？

答案：`event.stopPropagation()` 阻止冒泡，`event.preventDefault()` 阻止默认行为。`return false` 在 jQuery 中同时阻止两者（在原生 JS 中仅阻止默认行为而不阻止冒泡）。

2. JavaScript 进阶

Q: 什么是执行上下文？创建阶段包含什么？

答案：执行上下文是代码被解析和执行时的环境抽象。创建阶段：1. 确定 this（This Binding）；2. 创建词法环境；3. 创建变量环境。

Q: 闭包中的变量会被垃圾回收吗？

答案：如果闭包被全局变量引用则不会回收（从 Root 可达）。如果只是临时调用、无外部引用，执行完会回收。

Q: JavaScript 的垃圾回收算法有哪些？

答案：1. 引用计数（不再使用，循环引用问题）；2. 标记清除（主流算法，从根部扫描可达对象）。

Q: 四种常见的 JS 内存泄漏是什么？

答案：1. 意外的全局变量；2. 被遗忘的计时器/回调；3. 脱离 DOM 的引用；4. 闭包不当使用。

Q: V8 的内存限制是多少？为什么？

答案：64 位 1.4G，32 位 0.7G。因为垃圾回收一次需要暂停（Stop The World），内存太大会导致长时间停顿，影响性能。

Q: V8 的垃圾回收机制是什么？

答案：基于分代回收：新生代（Scavenge 算法，From/To 空间互换）和老生代（Mark-Sweep 标记清除 + Mark-Compact 标记整理）。通过增量标记（Incremental Marking）减少停顿。

Q: 什么是增量标记？

答案：将完整的标记过程拆分成多个小步骤，与程序运行交替进行，将停顿时间减少到 1/6 左右。

Q: 新生代对象何时晋升到老生代？

答案：1. 经过 5 次以上回收仍然存活；2. To 空间使用占比超过 25%；3. 超大对象。

Q: 为什么 const 声明的对象可以修改属性？

答案：const 保证的是变量指向的内存地址不变，而非内存地址中的数据不可变。对于引用类型，地址保存的是指针，指针不变但指向的对象内容可变。

Q: 从内存角度看 null 和 undefined 的区别？

答案：null 表示「空对象引用」，是一个赋值给变量的值，表示该变量不再指向任何对象。undefined 表示「未定义」，变量声明但未初始化时的默认值。typeof null 返回 'object'，typeof undefined 返回 'undefined'。

Q: 深度优先遍历和广度优先遍历如何实现？

答案：DFS 用栈（或递归），从一个节点出发追溯到底再回溯。BFS 用队列，先访问所有邻接节点再逐层深入。

Q: document.write 和 innerHTML 的区别？

答案：document.write 会重写整个页面，可用在页面加载时动态写入内容。innerHTML 修改指定元素内部 HTML，更精确。

Q: 什么是尾调用？有什么好处？

答案：函数的最后一步是调用另一个函数。好处：尾调用优化使调用栈不增加，可防止栈溢出。

3. ES6+

Q: ES6+ 新特性汇总？

答案：ES6：类、模块化、箭头函数、默认参数、模板字符串、解构赋值、扩展运算符、Promise、let/const、Symbol、Set/Map/WeakSet/WeakMap、Proxy/Reflect、Generator。ES7：Array.includes()、指数运算符 **。ES8：async/await、Object.values()/entries()、String padding。ES9：for await of、Promise.finally()、Rest/Spread 对象。ES10：flat/flatMap、Object.fromEntries、String.trimStart/trimEnd。ES11：可选链 ?.、空值合并 ??、Promise.allSettled、globalThis。ES12：replaceAll、Promise.any、逻辑赋值运算符 &&= || = ?? =。

Q: 箭头函数的 this 指向谁？

答案：继承外层普通函数的 this，无法通过 bind/call/apply 修改。

Q: 解构赋值的用途？

答案：快速提取对象/数组中的值：`const { name, age } = obj`、`const [a, b] = arr`。可用于函数参数默认值、交换变量值等。

Q: Map 和 Object 的区别？

答案：Map 的键可以是任意类型（包括对象），有序，可迭代，有 size 属性。Object 的键只能是字符串或 Symbol，无序（ES6 后部分有序），不可直接迭代。

Q: Symbol 的特性？

答案： 唯一且不可变，可用作对象属性的唯一键，防止属性名冲突。`Symbol()` 每次创建都不同，`Symbol.for('key')` 可创建全局 `Symbol`。

Q: 可选链 `?.` 和空值合并 `??` 的作用？

答案： `?.` 允许读取深层嵌套属性而不报错（短路返回 `undefined`）。`??` 只在值为 `null/undefined` 时返回右侧值，与 `||`（布尔假值即返回右侧）不同。

Q: `for...in` 和 `for...of` 的区别？

答案： `for...in` 遍历键（索引/属性名），适用于对象。`for...of` 遍历值，适用于可迭代对象（`Array`、`Set`、`Map`、`String` 等）。

Q: 什么是 Generator 函数？

答案： Generator 通过 `function*` 声明，使用 `yield` 暂停执行，通过 `.next()` 恢复执行。可实现状态机、异步流程控制。

Q: 什么是 Proxy？和 `Object.defineProperty` 对比？

答案： Proxy 可代理整个对象（包括数组、属性添加/删除），有 13 种拦截方法。`defineProperty` 只能劫持已有属性，无法监听数组变化和属性增删。

4. 异步编程

Q: 什么是 Promise？

答案： Promise 是异步编程的解决方案，是一个容器保存未来才结束的事件结果。三种状态：`pending`（进行中）、`fulfilled`（已成功）、`rejected`（已失败）。状态不可逆。

Q: Promise 解决了什么问题？

答案： 解决回调地狱（`callback hell`），使异步代码可读性更强，支持链式调用，提供统一异常处理。

Q: Promise 的 API 有哪些？

答案： `Promise.resolve/reject`、`Promise.all/race/allSettled/any`、`Promise.prototype.then/catch/finally`。

Q: `Promise.all` 和 `Promise.race` 的区别？

答案： `all` 等待所有 promise 完成，任一失败则整体失败。`race` 取第一个完成的 promise，无论成功或失败。

Q: `Promise.allSettled` 的作用？

答案： 等待所有 promise 完成（无论成功或失败），返回每个 promise 的结果（包含 `status` 和 `value/reason`）。

Q: `Promise.all` 和 `Promise.race` 传空数组会怎样？

答案： `Promise.all([])` 立即 resolve，结果是 `[]`。`Promise.race([])` 永远不决议（一直处于 `pending` 状态）。

Q: 如何中断 Promise 链?

答案: 在回调中返回一个 pending 状态的 Promise (`new Promise(() => {})`)。

Q: async/await 的原理?

答案: async 返回 Promise。await 等待 Promise 决议, 本质是 Generator + Promise 的语法糖, 内部实现了自动执行器。

Q: 宏任务 (MacroTask) 和微任务 (MicroTask) 有哪些?

答案: 宏任务: script (整体代码)、setTimeout、setInterval、I/O、UI rendering、setImmediate。微任务: Promise.then/catch/finally、MutationObserver、process.nextTick (Node)。

Q: 事件循环 (Event Loop) 的执行顺序?

答案: 1. 执行当前宏任务 (同步代码); 2. 执行所有微任务; 3. 尝试渲染 UI (浏览器); 4. 取出下一个宏任务执行。微任务队列清空后才取宏任务。

Q: setTimeout、Promise.then、async/await 的执行顺序面试题?

答案: 先执行同步代码, 再执行 Promise.then (微任务), 再执行 async 中 await 后的代码 (实际是 then 的语法糖), 最后执行 setTimeout (宏任务)。

Q: 传统的回调式异步有什么缺点?

答案: 回调地狱、信任问题 (调用太早/太晚/次数不可控、吞掉异常、未能传递所需参数)。

Q: 异步解决方案的发展历程?

答案: 回调函数 → Promise → Generator → async/await。每个阶段解决前一阶段的问题: Promise 解决回调地狱, Generator 可控制执行, async/await 是终极方案。

5. HTML

Q: DOCTYPE 的作用?

答案: 告知浏览器以何种文档标准解析页面。标准模式按最高标准渲染, 兼容模式模拟老浏览器行为。HTML5 只需 `<!DOCTYPE html>`。

Q: HTML5 有哪些新特性?

答案: 语义化标签 (header/footer/article/section/nav)、canvas/SVG、video/audio、Web Storage (localStorage/sessionStorage)、webworker、websocket、Geolocation、FormData、拖放 API。

Q: 什么是语义化? 有什么好处?

答案: 用正确的标签做正确的事。好处: 结构清晰, 便于浏览器和搜索引擎解析, 利于 SEO, 无 CSS 时也能文档化显示, 便于维护。

Q: meta viewport 的作用?

答案： 设置视口大小以适应移动设备。常用：`<meta name="viewport" content="width=device-width, initial-scale=1.0">`。

Q: img 的 title 和 alt 的区别？

答案： title 鼠标悬停时显示。alt 是图片内容的等价描述，图片加载失败时显示，被读屏软件读取，利于 SEO。

Q: 前端的 SEO 优化有哪些？

答案： 合理设置 title/description/keywords；语义化 HTML；重要内容放前面；少用 iframe；图片加 alt；提高网站速度；使用 SSR。

Q: iframe 的缺点？

答案： 阻塞主页面 onload；不利于 SEO；与主页面共享连接池；管理复杂。

Q: 浏览器内多标签页之间如何通信？

答案： 1. localStorage（监听 storage 事件）；2. postMessage；3. WebSocket；4. SharedWorker；5. BroadcastChannel API。

Q: Canvas 和 SVG 的区别？

答案： Canvas 基于像素（位图），通过 JS 绘制，适合游戏/密集图形，放大失真。SVG 基于矢量（XML），DOM 操作，适合图标/图表，放大不失真。

6. CSS

Q: 盒模型有哪些？

答案： W3C 标准盒模型（content-box）：width/height 只包含 content。IE 盒模型（border-box）：width/height 包含 content + padding + border。通过 `box-sizing` 控制。

Q: BFC 是什么？如何创建？

答案： 块级格式化上下文，独立渲染区域。创建方式：根元素、float、absolute/fixed、overflow 不为 visible、inline-block/flex/table-cell。

Q: BFC 的作用有哪些？

答案： 1. 清除浮动（包含浮动元素）；2. 防止 margin 折叠；3. 阻止元素被浮动元素覆盖（自适应两栏布局）。

Q: CSS 选择器的优先级？

答案： `!important` > 行内样式（1000）> ID（100）> 类/属性/伪类（10）> 元素/伪元素（1）> 通配符（0）。权重按位相加比较。

Q: position 各值的区别？

答案：static（默认）→ 正常文档流。relative → 相对自身原位置偏移，原空间保留。absolute → 相对最近非static 定位的祖先元素，脱离文档流。fixed → 相对视口。sticky → 粘性定位，相对视口和父元素混合。

Q: 水平垂直居中的方式？

答案：flex: `justify-content: center; align-items: center`。绝对定位 + `margin:auto`。绝对定位 + `transform:translate(-50%,-50%)`。table-cell + `vertical-align:middle` + `text-align:center`。

Q: display:none、visibility:hidden、opacity:0 的区别？

答案：display:none 从渲染树消失，不占空间，不响应事件。visibility:hidden 占据空间，不响应事件，子元素可设 visible 显示。opacity:0 占据空间，响应事件。

Q: Flex 布局核心属性？

答案：容器：flex-direction、flex-wrap、justify-content、align-items、align-content。项目：order、flex-grow、flex-shrink、flex-basis、align-self。

Q: 如何使用 CSS 实现三角形？

答案：宽高设为 0，border 设为所需宽度，三边颜色透明，一边设颜色。

Q: CSS 清除浮动的方式？

答案：1. 伪元素 `::after { content: ''; display: table; clear: both; }`；2. 父元素 `overflow:hidden`（触发 BFC）；3. 父元素也浮动；4. 结尾加空 `div clear:both`。

Q: CSS 选择器的匹配规则？

答案：从右向左匹配（从关键选择器开始），这样可以快速排除不匹配的元素，避免无效匹配。

Q: link 和 @import 的区别？

答案：link 是 HTML 标签，@import 是 CSS 方式。link 页面加载时同时加载，@import 页面加载完后才加载。link 兼容性更好，支持 JS 操作。

Q: 什么是 CSS Sprite（雪碧图）？

答案：将多个小图标合并到一张图片，通过 background-position 定位显示。优点：减少 HTTP 请求。缺点：维护麻烦。

Q: px / em / rem / vw / vh 的区别？

答案：px 绝对单位。em 相对父元素字体大小。rem 相对根元素（html）字体大小。vw 视口宽度的 1%。vh 视口高度的 1%。

Q: transition 和 animation 的区别？

答案：transition 需要触发条件（如 hover），只有开始和结束状态。animation 不需要触发，可用 @keyframes 定义多关键帧动画，更灵活。

Q: 响应式设计的原理？

答案：通过 CSS3 媒体查询 `@media` 检测设备屏幕尺寸，针对不同宽度加载不同样式。结合 `rem/vw` 等相对单位和 `flex/grid` 布局。

Q: CSS 中哪些属性可以继承？

答案：字体系列 (`font`、`font-size`、`font-weight` 等)、文本系列 (`color`、`text-align`、`text-indent` 等)、`visibility`、`cursor` 等。

Q: 三栏布局（两边固定中间自适应）的实现方式？

答案：`flex`（左右固定宽，中间 `flex:1`）；圣杯布局（浮动+负边距+`padding`）；双飞翼布局（浮动+负边距+`margin`）；绝对定位。

7. 浏览器原理

Q: 从输入 URL 到页面呈现经历了什么？

答案：网络阶段：1. 构建请求行 → 2. 查找强缓存 → 3. DNS 解析 → 4. TCP 三次握手 → 5. 发送 HTTP 请求 → 6. 服务器处理并返回响应 → 7. TCP 四次挥手/持久连接。渲染阶段：1. 解析 HTML 构建 DOM 树 → 2. 解析 CSS 构建 CSSOM 树 → 3. 合并为渲染树 → 4. 布局 (Layout) → 5. 分层 (Layer) → 6. 绘制 (Paint) → 7. 分块栅格化 → 8. 合成显示。

Q: 浏览器渲染进程包含哪些线程？

答案：1. GUI 渲染线程（渲染界面，与 JS 引擎互斥）；2. JS 引擎线程（执行 JS）；3. 事件触发线程（控制事件循环）；4. 定时器触发线程（处理 `setTimeout/setInterval`）；5. 异步 HTTP 请求线程。

Q: 为什么 JS 是单线程的？

答案：最初设计基于 DOM 操作需求。多线程同时操作 DOM 会导致数据不一致。浏览器设置 GUI 渲染线程与 JS 引擎互斥。

Q: CSS 加载会阻塞 DOM 解析和渲染吗？

答案：CSS 不阻塞 DOM 解析，但阻塞 DOM 渲染（渲染树需 DOM + CSSOM）。CSS 阻塞其后的 JS 执行（JS 可能依赖样式）。

Q: 为什么 JS 会阻塞页面加载？

答案：JS 可能操作 DOM，浏览器不确定 JS 执行结果，会等 JS 执行完再继续 DOM 解析。GUI 渲染线程与 JS 引擎互斥。

Q: `defer` 和 `async` 的区别？

答案：两者都异步加载，不阻塞 HTML 解析。`async` 加载完立即执行，不保证顺序。`defer` 在文档解析完成后、`DOMContentLoaded` 前按序执行。

Q: `DOMContentLoaded` 和 `load` 事件的区别？

答案：DOMContentLoaded 在 DOM 解析完成后触发（无需等待样式表、图片）。load 在所有资源（包括图片、样式）加载完后触发。

Q: 什么是重排（Reflow）和重绘（Repaint）？

答案：重排：DOM 几何属性改变，浏览器重新计算元素位置和尺寸，从重新构建渲染树开始。重绘：外观改变不影响布局（color/background），只重新绘制。重排必重绘，重绘不一定重排。

Q: 触发重排的操作有哪些？

答案：1. 增删可见 DOM；2. 改变元素位置/尺寸；3. 改变窗口大小；4. 改变字体；5. 激活 CSS 伪类；6. 查询 offsetTop/scrollTop/getComputedStyle 等（浏览器 flush 队列）。

Q: 如何优化重排重绘？

答案：1. 使用 transform 替代 top/left；2. 离线 DOM（display:none 后修改）；3. 类名批量修改样式；4. DocumentFragment 批量操作；5. 避免循环读取 offset 属性；6. 动画元素使用 absolute/fixed 脱离文档流；7. 启用 GPU 加速（transform:translateZ(0)）；8. 使用 rAF。

Q: 浏览器进程有哪些？

答案：1. Browser 进程（协调主控，只一个）；2. 渲染进程（每个 Tab 一个）；3. GPU 进程；4. 网络进程；5. 第三方插件进程。

Q: 普通图层和复合图层有什么区别？

答案：默认文档流属于同一个复合图层。通过硬件加速（translate3d/opacity/will-change）可创建新的复合图层，由 GPU 独立绘制，不触发其他图层重排。但不要过多使用，否则资源消耗大。

Q: Web Worker 和 SharedWorker 的区别？

答案：Web Worker 属于某个页面，在渲染进程中创建新线程运行 JS。SharedWorker 浏览器所有页面共享，为它单独创建进程。本质是线程与进程的区别。

Q: 什么是 requestAnimationFrame？

答案：告诉浏览器在下次重绘前调用指定函数，由系统决定执行时机（通常 60Hz）。页面隐藏时自动停止，节省 CPU 开销。

8. 网络协议

Q: HTTP 和 HTTPS 的区别？

答案：HTTP 明文传输，端口 80。HTTPS 通过 SSL/TLS 加密传输，端口 443。HTTPS 需要 CA 证书，安全性更高，但速度略慢。

Q: HTTP 常见的状态码？

答案：1xx：信息（101 Switching Protocols）；2xx：成功（200 OK，201 Created，204 No Content，206 Partial Content）；3xx：重定向（301 永久重定向，302 临时重定向，304 未修改，307 临时重定向且不允许改

变请求方法)；4xx：客户端错误（400 参数错误，401 未授权，403 禁止，404 未找到，405 方法不允许，429 请求过多）；5xx：服务器错误（500 内部错误，502 网关错误，503 服务不可用，504 网关超时）。

Q: HTTP 有哪些请求方法？

答案：GET（获取资源）、POST（提交数据）、PUT（更新/替换资源）、DELETE（删除资源）、PATCH（部分修改资源）、HEAD（只获取响应头）、OPTIONS（获取支持的请求方法，用于 CORS 预检请求）、CONNECT（建立隧道，用于 HTTPS 代理）。

Q: GET 和 POST 的区别？

答案：语义上 GET 获取资源（安全、幂等），POST 提交数据（不安全、不幂等）。GET 参数在 URL 上，可缓存和收藏，有长度限制，只 ASCII 字符。POST 参数在请求体，不可缓存，无长度限制，支持二进制。安全性上 POST 略好（参数不暴露在 URL）。注意：GET 和 POST 都可能产生一个或两个 TCP 包（取决于浏览器实现），这不是本质区别。

Q: TCP 三次握手的过程？

答案：1. 客户端发送 SYN (seq=x)，进入 SYN_SENT；2. 服务端回复 SYN+ACK (seq=y, ack=x+1)，进入 SYN_RCVD；3. 客户端发送 ACK (seq=x+1, ack=y+1)，进入 ESTABLISHED。

Q: TCP 四次挥手的过程？

答案：1. 客户端发送 FIN；2. 服务端回复 ACK；3. 服务端发送 FIN；4. 客户端回复 ACK（等待 2MSL 后关闭）。

Q: 为什么三次握手而不是两次？

答案：防止已失效的连接请求突然传到服务器产生错误。三次握手才能确认双方的收发能力都正常。

Q: 什么是 SYN 泛洪攻击？

答案：攻击者发送大量 SYN 包（第一次握手成功），服务器回复 SYN+ACK（第二次握手），但攻击者不回应 ACK，导致服务器大量半连接占用资源。防御：SYN Cookie，第二次握手时不创建半连接。

Q: HTTP/1.0 和 HTTP/1.1 的区别？

答案：HTTP/1.1 默认持久连接（keep-alive），支持管道化，增加 Host 头（虚拟主机），支持断点续传（Range），新增 24 个错误状态码。

Q: HTTP/2 对比 HTTP/1.1 有哪些新特性？

答案：1. 二进制分帧（而非文本传输）；2. 多路复用（一个 TCP 连接承载多个请求）；3. Header 压缩（HPACK 算法）；4. 服务端推送（Server Push）；5. 流量控制。但仍存在 TCP 队头阻塞（丢包时整个 TCP 等待）。

Q: HTTP/3 和 QUIC 为什么选择 UDP？

答案：基于 TCP 修改兼容困难，UDP 无连接、无建链拆链成本、无队头阻塞问题。QUIC 特性：0-RTT 建连、多路复用无队头阻塞、向前纠错机制、加密认证。

Q: HTTPS 握手过程？

答案：1. 客户端发送支持的 SSL 版本和加密套件；2. 服务器选择加密套件并返回证书；3. 客户端验证证书，生成预主密钥并用服务器公钥加密发送；4. 服务器用私钥解密，双方生成会话密钥；5. 后续使用对称加密通信。

Q: Cookie、localStorage、sessionStorage 的区别？

答案：Cookie 大小 4KB，每次请求自动携带，可设过期时间，默认会话级。localStorage 约 5MB，持久存储，不随请求发送。sessionStorage 约 5MB，关闭标签页即清除。Web Storage 有 setItem/getItem/removeItem API。

Q: Cookie 的属性有哪些？

答案：Name/Value、Domain、Path、Expires/Max-Age、Secure（仅 HTTPS）、HttpOnly（防 XSS）、SameSite（Strict/Lax/None，防 CSRF）。

Q: Session 和 Cookie 的关系？

答案：Session 存储在服务端，Cookie 存储在客户端。SessionID 通过 Cookie 传输。服务端通过 SessionID 找到对应的 Session 数据。Session 比 Cookie 安全。

Q: 浏览器缓存机制是怎样的？

答案：强缓存：Expires（HTTP/1.0，绝对时间）和 Cache-Control（HTTP/1.1，相对时间），命中则直接从缓存读取（200 from cache），不发请求。协商缓存：Last-Modified/If-Modified-Since 和 ETag/If-None-Match，向服务器验证资源是否更新，未更新返回 304。Cache-Control 优先级高于 Expires，ETag 优先级高于 Last-Modified。

Q: Cache-Control 有哪些常用指令？

答案：max-age（缓存秒数）、s-maxage（代理缓存）、public/private（是否可代理缓存）、no-cache（使用协商缓存）、no-store（不使用任何缓存）、must-revalidate（过期后必须验证）。

Q: DNS 解析的完整过程？

答案：浏览器缓存 → 操作系统缓存（hosts 文件）→ 本地 DNS 服务器（递归查询）→ 根域名服务器 → 顶级域名服务器（如 .com）→ 权威域名服务器 → 返回 IP。DNS 主要使用 UDP（端口 53），超过 512 字节时用 TCP。

Q: HTTP keep-alive 和多路复用的区别？

答案：HTTP/1.x keep-alive 保持 TCP 连接，但同一时刻一个连接只能处理一个请求（串行）。HTTP/2 多路复用可在同一连接中并发多个请求和响应，通过二进制分帧交错发送。

Q: WebSocket 是什么？

答案：应用层协议，实现客户端与服务端全双工通信。通过 HTTP 升级（Upgrade header）建立连接。数据包小，无同源限制，有 open/message/error/close 事件。

Q: CDN 的工作原理？

答案：将源站内容缓存到全球各地节点服务器。用户请求时，通过 DNS 智能解析将域名指向最近的 CDN 节点。可提高访问速度、减轻源站压力。

Q: 什么是 Cookie 隔离？

答案：静态资源放在 CDN 的非主要域名下，请求头不携带主域名的 Cookie，减少请求头大小，加快请求速度。

Q: TCP 和 UDP 的区别？

答案：TCP 面向连接、可靠、有序、有流量控制和拥塞控制，速度慢。UDP 无连接、不可靠、无序，速度快，适合实时应用（视频/游戏）。

9. 性能优化

Q: 前端性能优化有哪些方面？

答案：加载优化：减少 HTTP 请求（雪碧图、合并文件）、CDN、Gzip、缓存策略、懒加载。渲染优化：CSS 放 head、JS 放 body 底部、减少重排重绘、CSS3 代替 JS 动画、离屏渲染。构建优化：Tree Shaking、代码分割、DllPlugin、压缩。图片优化：WebP、响应式图片、Base64 小图标、图片懒加载。

Q: 什么是防抖（Debounce）和节流（Throttle）？

答案：防抖：持续触发事件时，每次重新计时，只在最后一次触发后等待指定时间才执行。节流：持续触发事件时，固定时间内只执行一次。防抖适合搜索输入，节流适合滚动/resize。

Q: 图片懒加载如何实现？

答案：判断图片进入可视区时，将 data-src 属性值赋给 src。判断方式：`getBoundingClientRect().top < window.innerHeight`。配合节流优化 scroll 事件。

Q: 如何优化首屏加载速度？

答案：1. 路由懒加载（代码分割）；2. 静态资源使用 CDN；3. 开启 Gzip；4. 首屏关键 CSS 内联；5. 图片懒加载；6. 使用 SSR（服务端渲染）；7. 减少首屏 JS 体积（Tree Shaking / external）。

Q: 服务端渲染（SSR）的原理和优缺点？

答案：将组件在服务端渲染成 HTML 字符串，直接返回给浏览器。优点：SEO 友好、首屏加载快。缺点：服务器负载增加、开发复杂度提升、需要 Node.js 环境。

Q: WebP 图片格式有何优势？

答案：Google 开发，支持有损/无损压缩，比 PNG 小 26%，比 JPEG 小 25-34%，支持透明和动图。兼容性有限（Safari 部分版本不支持）。

Q: 为什么 CSS 动画比 JS 动画高效？

答案：CSS 动画可能只触发合成（composite），不触发重排重绘，由 GPU 处理。JS 动画容易触发重排。transform 和 opacity 是仅触发合成的属性。

10. Vue

Q: Vue 响应式原理是什么？

答案：通过 `Object.defineProperty` 劫持 `data` 各属性的 `getter/setter`。每个属性对应一个 `dep`（依赖收集器），模板中每个表达式对应一个 `watcher`。数据变化时，`dep` 通知所有 `watcher` 更新视图。Vue3 改用 `Proxy` 代理整个对象。

Q: Vue 组件中 `data` 为什么必须是函数？

答案：组件可被多次复用。如果 `data` 是对象，多个实例共享同一 `data`，造成数据污染。函数返回新对象，每个实例独立。根实例只有一个，所以可以是对象。

Q: Vue 生命周期有哪些？

答案：创建：`beforeCreate` → `created`（数据观测完成）。挂载：`beforeMount` → `mounted`（DOM 挂载完成）。更新：`beforeUpdate` → `updated`。销毁：`beforeDestroy`（清理定时器/事件）→ `destroyed`。还有 `activated/deactivated`（`keep-alive`）。

Q: Vue 组件通信方式有哪些？

答案：1. `props/$emit`（父子）；2. `$bus`（事件总线，兄弟/跨级）；3. `Vuex`（全局状态管理）；4. `provide/inject`（祖先后代）；5. `$attrs/$listeners`；6. `slot`（插槽）。

Q: `v-if` 和 `v-show` 的区别？

答案：`v-if` 条件性渲染，为 `false` 时元素不存在于 DOM。`v-show` 切换 `display` 属性，元素始终存在于 DOM。频繁切换用 `v-show`，运行条件很少改变用 `v-if`。

Q: `v-if` 和 `v-for` 哪个优先级更高？

答案：`v-for` 优先级高于 `v-if`（Vue2），每次渲染先循环再判断条件。优化：在外层 `template` 用 `v-if`，内部再用 `v-for`；或通过计算属性提前过滤。

Q: Vue 中 `key` 的作用？

答案：`key` 给 `vnode` 唯一标识。在 `diff` 中通过 `key` 精准判断节点是否为同一节点，避免不必要的 DOM 更新，提高性能。不设 `key` 可能引发隐藏的 bug。

Q: Vue 的 `diff` 算法是怎样的？

答案：深度优先、同层比较。先假设头尾节点可能相同（4 次尝试），没找到则用 `key` 遍历查找。找到相同节点则复用并更新，不同则创建新节点。只比较同级，不跨层比较。

Q: `computed` 和 `watch` 的区别？

答案：`computed` 基于依赖缓存，只有依赖变化才重新计算，不支持异步。`watch` 监听数据变化执行回调，支持异步，可执行复杂逻辑。能用 `computed` 的尽量用 `computed`。

Q: `Vuex` 的核心概念？

答案：`state`（状态）、`getters`（派生状态）、`mutations`（同步修改状态）、`actions`（异步操作，提交 `mutation`）、`modules`（模块化）。严格模式下一律通过 `action` → `mutation` 修改 `state`。

Q: 为什么 mutations 不能做异步操作?

答案: 异步操作使状态变化不可追踪, 无法保证可预测性和可调试性。mutation 必须是同步纯函数, 异步在 action 中处理。

Q: Vue 中如何监听对象属性的变化?

答案: 使用 watch: `watch: { 'obj.a': { handler() {} } }` 或 `{ deep: true }` 深度监听。新增属性用 `Vue.set()` / `this.$set()`。

Q: Vue 中 nextTick 的原理?

答案: Vue 异步更新 DOM, 数据变化后缓冲同一事件循环中所有变更。nextTick 在 DOM 更新后执行回调, 优先 Promise.then → MutationObserver → setTimeout。

Q: Vue Router 传参方式?

答案: 1. 动态路由 (params): `/user/:id, $route.params.id`; 2. query: `?id=1, $route.query.id`。

Q: keep-alive 的作用?

答案: 缓存组件实例, 避免重复创建销毁。被缓存组件有 activated (唤醒) 和 deactivated (休眠) 生命周期钩子。

Q: Vuex 数据刷新丢失如何处理?

答案: 数据存储在内存中。方案: 1. 每次加载重新请求; 2. 订阅 Vuex 变化同步到 sessionStorage; 3. 在 beforeunload 事件中保存数据到存储。

Q: Vue 性能优化方法?

答案: 路由懒加载、keep-alive 缓存、v-show 复用 DOM、v-for 避免同时用 v-if、长列表用 Object.freeze/虚拟滚动、事件销毁、图片懒加载、第三方插件按需引入、函数式组件。

Q: Vue3 相比 Vue2 的改进?

答案: 使用 Proxy 代替 Object.defineProperty (支持数组和属性增删); Composition API 代替 Options API; Fragments (多根节点); Teleport; 更好的 TypeScript 支持; 性能提升 1.3-2 倍。

11. React

Q: 虚拟 DOM 为什么能提高性能?

答案: 虚拟 DOM 是 JS 对象结构表示的 DOM 树。在 JS 和真实 DOM 间加缓存, 通过 diff 算法找出最小变更, 批量更新真实 DOM。首次渲染不一定更快, 优势在于后续更新的效率。

Q: setState 是同步还是异步?

答案: 在 React 合成事件和生命周期中是异步的 (批量更新)。在 setTimeout、原生 DOM 事件中是同步的。

Q: React 组件的生命周期?

答案：挂载：constructor → getDerivedStateFromProps → render → componentDidMount。更新：getDerivedStateFromProps → shouldComponentUpdate → render → getSnapshotBeforeUpdate → componentDidUpdate。卸载：componentWillUnmount。

Q: React 性能优化在哪个生命周期？

答案：shouldComponentUpdate，判断是否需要重新渲染。React.memo（函数组件）和 PureComponent（类组件）是浅比较实现。

Q: React Hooks 是什么？

答案：React 16.8 引入，让函数组件拥有状态管理。常用：useState（状态）、useEffect（副作用）、useContext（上下文）、useRef（引用）、useMemo（记忆值）、useCallback（记忆函数）、useReducer（复杂状态）。

Q: Hooks 相比类组件的优势？

答案：代码更简洁，没有 this 绑定问题，没有生命周期方法的繁琐，通过自定义 Hooks 轻松复用状态逻辑，逻辑关注点更集中。

Q: React 中 key 的作用？

答案：标识虚拟 DOM 对象。在 diff 中通过 key 判断节点是否可复用。key 改变时销毁旧 DOM 创建新 DOM；key 不变且数据没变则复用；key 不变且数据变则更新。

Q: 高阶组件（HOC）是什么？

答案：以组件为参数返回新组件的函数。用于复用代码逻辑，如 Redux 的 connect。

Q: React 合成事件是什么？

答案：为解决跨浏览器兼容性，React 将原生事件封装为合成事件。采用事件委托，将事件绑定到根元素上。

Q: 受控组件和非受控组件的区别？

答案：受控组件：表单 value 由 React state 控制，通过 onChange 更新。非受控组件：表单状态由 DOM 自身维护，通过 ref 获取值。

Q: Redux 的核心概念？

答案：store（单一状态树）、action（描述事件）、reducer（纯函数，返回新状态）、dispatch（派发 action）、middleware（中间件，处理异步）。

Q: Redux 为什么不能做异步操作？

答案：Reducer 必须是纯函数，确保状态变化的可预测性。异步操作通过中间件（redux-thunk/redux-saga）处理。

Q: React diff 算法的策略？

答案：1. 同级比较（跨层级不比较）；2. 不同类型节点直接替换；3. 同类型节点通过 key 匹配更新。

12. 工程化

Q: webpack 的核心概念有哪些？

答案：Entry（入口）、Output（输出）、Loader（模块转换）、Plugin（插件扩展）、Mode（development/production）。

Q: Loader 和 Plugin 的区别？

答案：Loader 处理模块转换（webpack 原生只懂 JS），在 module.rules 配置。Plugin 扩展 webpack 功能（打包优化、资源管理），在 plugins 配置。

Q: webpack 的构建流程？

答案：1. 初始化参数；2. 创建 Compiler 对象加载插件；3. 从 Entry 开始递归构建依赖图；4. 调用 Loader 转换模块；5. 组装 Chunk；6. 输出文件。

Q: 什么是 HMR（热模块替换）？

答案：模块热替换，开发环境修改代码后只替换变更模块，无需刷新整个页面。保持应用状态。

Q: 如何优化 webpack 构建速度？

答案：1. 缩小 Loader 范围（include/exclude）；2. 多进程打包（thread-loader）；3.DllPlugin 预打包第三方库（Webpack 5 推荐用 HardSourceWebpackPlugin 或 cache 配置替代）；4. 开启 babel 缓存；5. resolve.alias 缩短路径查找；6. externals 避免打包某些库。

Q: 什么是 Tree Shaking？

答案：删除未引用的代码（死代码消除）。依赖 ES6 模块的静态结构特性。生产模式下自动开启。

Q: Webpack 如何做代码分割（Code Split）？

答案：配置 optimization.splitChunks。将 node_modules 单独打包、多入口公共部分提取、路由懒加载（动态 import）。

Q: 模块化方案有哪些？

答案：1. IIFE（早期方案）；2. AMD/CMD（浏览器异步加载）；3. CommonJS（Node 同步加载，require/module.exports）；4. ES Module（原生支持，import/export）。

Q: CommonJS 和 ES Module 的区别？

答案：CommonJS 同步加载、值拷贝（导入值不变）。ES Module 异步加载、实时绑定（导入值随导出变化）。CommonJS 支持动态 require，ES Module 静态结构（支持 Tree Shaking）。

Q: Babel 的原理？

答案：源码 → 解析（parse）为 AST → 转换（transform）新 AST → 生成（generate）目标代码。插件和预设（preset）控制转换规则。

Q: Git Flow 常用的分支策略？

答案： master（生产）、develop（开发）、feature/（新功能）、release/（发布准备）、hotfix/*（紧急修复）。功能开发在 feature 分支，完成后合并到 develop。

13. TypeScript 与 Axios

Q: TypeScript 相比 JavaScript 的优势？

答案： 1. 静态类型检查，编译阶段发现类型错误；2. 更好的 IDE 支持和代码提示；3. 接口（interface）和类型别名（type）让代码自文档化；4. 泛型支持提高代码复用性和类型安全；5. 面向对象特性（类、抽象类、修饰符）更完善；6. 大型项目维护成本更低。

Q: TypeScript 中的 interface 和 type 有什么区别？

答案： 1. interface 可以重复声明（合并），type 不行；2. interface 可以被 class implements，type 不行；3. type 可以定义联合类型/交叉类型/元组等复杂类型；4. 能用 interface 的地方优先用 interface，否则用 type。

Q: TypeScript 中的泛型是什么？

答案： 泛型（Generics）指在定义函数/接口/类时不预先指定具体类型，使用时再指定类型的特性。如 `function identity<T>(arg: T): T { return arg }`。提高代码复用性和类型安全。

Q: TypeScript 中的工具类型有哪些？

答案： Partial（所有属性可选）、Required（所有属性必选）、Readonly（所有属性只读）、Pick（选取部分属性）、Omit（排除部分属性）、Record（构造对象类型）、Exclude（联合类型排除）、ReturnType（获取函数返回类型）。

Q: Axios 拦截器的执行顺序？

答案： 请求拦截器后添加先执行（类似栈），响应拦截器先添加先执行（类似队列）。整体顺序：请求拦截器 2 → 请求拦截器 1 → 发送请求 → 响应拦截器 1 → 响应拦截器 2 → 请求回调。通过 Promise 链串连。

Q: Axios 如何取消请求？

答案： 使用 CancelToken： `const source = axios.CancelToken.source()`，将 `source.token` 传给 config，需要取消时调用 `source.cancel('message')`。在新版本 axios 中推荐使用 AbortController： `new AbortController()` + `signal` 属性。

Q: Axios 与 Axios.create 的关系？

答案： axios 本身是 `Axios.prototype.request` 函数 bind 后的结果，既有函数特性又具备 Axios 实例的方法。`axios.create(config)` 创建一个新的 axios 实例，有自己的 defaults 和 interceptors，但没有 create/CancelToken/all 等静态方法。

14. 安全

Q: 什么是 XSS 攻击？如何防范？

答案： 跨站脚本攻击，注入恶意脚本到页面。三种类型：存储型（数据库持久化）、反射型（URL 参数）、DOM 型（纯客户端）。防范：输入过滤转义、CSP（内容安全策略）、HttpOnly Cookie。

Q: 什么是 CSRF 攻击？如何防范？

答案： 跨站请求伪造，利用用户登录状态发起恶意请求。防范：SameSite Cookie 属性（Strict/Lax）、验证 Referer/Origin、CSRF Token、验证码。

Q: 什么是中间人攻击？

答案： 攻击者拦截并篡改通信双方的数据。HTTPS 通过证书验证和加密通信可有效防止中间人攻击。

Q: 什么是 CSP？

答案： 内容安全策略，通过 HTTP 头 Content-Security-Policy 设置允许加载的资源来源，有效防止 XSS。

Q: 同源策略是什么？

答案： 协议、域名、端口都相同即为同源。同源策略限制不同源的文档/脚本对当前源数据的读取和设置，是浏览器最基本的安全功能。

Q: 跨域的解决方案有哪些？

答案： 1. JSONP（script 标签，只支持 GET）；2. CORS（服务端设置 Access-Control-Allow-Origin）；3. nginx 反向代理；4. postMessage（iframe 通信）；5. WebSocket（无同源限制）；6. document.domain（子域跨域）。

Q: HTTPS 的 CA 证书验证过程？

答案： 客户端获取服务器证书，通过层级关系找到上级 CA 证书，用上级公钥解密当前证书的数字签名，比对证书指纹是否一致。验证通过则证书可信。

Q: 什么是 Cookie 的 HttpOnly 属性？

答案： 设置后 JS 无法通过 document.cookie 访问 Cookie，可有效防止 XSS 窃取 Cookie。

Q: 什么是 SameSite 属性？

答案： Cookie 的 SameSite 属性控制跨站请求是否携带 Cookie。Strict（完全禁止跨站）、Lax（部分允许，GET 等安全请求）、None（不限制，需配合 Secure）。

15. 手写代码

Q: 实现防抖（Debounce）

```
function debounce(fn, delay) {  
  let timer = null  
  return function (...args) {  
    clearTimeout(timer)  
    timer = setTimeout(() => fn.apply(this, args), delay)  
  }  
}
```

```
}  
}
```

Q: 实现节流 (Throttle)

```
function throttle(fn, delay) {  
  let timer = null  
  return function (...args) {  
    if (!timer) {  
      timer = setTimeout(() => {  
        fn.apply(this, args)  
        timer = null  
      }, delay)  
    }  
  }  
}
```

Q: 实现深拷贝 (考虑循环引用和 Symbol)

```
function deepClone(obj, map = new WeakMap()) {  
  if (obj === null || typeof obj !== 'object') return obj  
  if (map.has(obj)) return map.get(obj)  
  const clone = Array.isArray(obj) ? [] : {}  
  map.set(obj, clone)  
  for (const key of Reflect.ownKeys(obj)) {  
    clone[key] = deepClone(obj[key], map)  
  }  
  return clone  
}
```

Q: 实现 call

```
Function.prototype.myCall = function (ctx, ...args) {  
  ctx = ctx ?? window  
  const key = Symbol()  
  ctx[key] = this  
  const result = ctx[key](...args)  
  delete ctx[key]  
  return result  
}
```

Q: 实现 apply

```
Function.prototype.myApply = function (ctx, args) {  
  ctx = ctx ?? window  
  const key = Symbol()  
  ctx[key] = this  
  const result = ctx[key](...args)  
  delete ctx[key]  
  return result  
}
```

Q: 实现 bind

```
Function.prototype.myBind = function (ctx, ...args) {  
  const fn = this  
  return function (...args2) {  
    return fn.apply(ctx, [...args, ...args2])  
  }  
}
```

Q: 实现 new

```
function myNew(Con, ...args) {  
  const obj = Object.create(Con.prototype)  
  const result = Con.apply(obj, args)  
  return result !== null && (typeof result === 'object' || typeof result ===  
    'function')  
    ? result : obj  
}
```

Q: 实现 instanceof

```
function myInstanceOf(obj, Con) {  
  let proto = obj.__proto__  
  while (proto) {  
    if (proto === Con.prototype) return true  
    proto = proto.__proto__  
  }  
  return false  
}
```

Q: 实现 Promise.all

```
Promise.myAll = function (promises) {  
  return new Promise((resolve, reject) => {
```

```
const results = []
let count = 0
for (let i = 0; i < promises.length; i++) {
  Promise.resolve(promises[i]).then(val => {
    results[i] = val
    count++
    if (count === promises.length) resolve(results)
  }, reject)
}
})
}
```

Q: 实现 Promise.race

```
Promise.myRace = function (promises) {
  return new Promise((resolve, reject) => {
    for (const p of promises) {
      Promise.resolve(p).then(resolve, reject)
    }
  })
}
```

Q: 实现 Promise.allSettled

```
Promise.myAllSettled = function (promises) {
  return Promise.all(promises.map(p =>
    Promise.resolve(p).then(
      value => ({ status: 'fulfilled', value }),
      reason => ({ status: 'rejected', reason })
    )
  ))
}
```

Q: 实现 Promise.any

```
Promise.myAny = function (promises) {
  return new Promise((resolve, reject) => {
    const errors = []
    let count = 0
    for (let i = 0; i < promises.length; i++) {
      Promise.resolve(promises[i]).then(resolve, err => {
        errors[i] = err
        count++
        if (count === promises.length) reject(new AggregateError(errors))
      })
    }
  })
}
```



```
  })  
}
```

Q: 实现 Promise.finally

```
Promise.prototype.myFinally = function (cb) {  
  return this.then(  
    val => Promise.resolve(cb()).then(() => val),  
    err => Promise.resolve(cb()).then(() => { throw err })  
  )  
}
```

Q: 实现节流（时间戳版）

```
function throttle(fn, delay) {  
  let prev = 0  
  return function (...args) {  
    const now = Date.now()  
    if (now - prev > delay) {  
      prev = now  
      return fn.apply(this, args)  
    }  
  }  
}
```

Q: 柯里化函数

```
function curry(fn) {  
  return function curried(...args) {  
    if (args.length >= fn.length) {  
      return fn.apply(this, args)  
    }  
    return (...args2) => curried(...args, ...args2)  
  }  
}
```

Q: 函数组合（compose）

```
function compose(...fns) {  
  return x => fns.reduceRight((acc, fn) => fn(acc), x)  
}
```

Q: 实现数组 flat

```
function flat(arr, depth = 1) {
  if (depth === 0) return arr
  return arr.reduce((acc, val) =>
    acc.concat(Array.isArray(val) ? flat(val, depth - 1) : val), [])
}
```

Q: 实现 EventEmitter (发布订阅)

```
class EventEmitter {
  constructor() { this.events = {} }
  on(event, cb) { (this.events[event] || []).push(cb); return this }
  emit(event, ...args) { this.events[event]?.forEach(cb => cb(...args)); return this }
  off(event, cb) {
    if (!cb) { delete this.events[event]; return this }
    this.events[event] = this.events[event]?.filter(fn => fn !== cb)
    return this
  }
  once(event, cb) {
    const wrapper = (...args) => { cb(...args); this.off(event, wrapper) }
    this.on(event, wrapper)
    return this
  }
}
```

Q: 实现 async/await

```
function asyncToGenerator(generatorFn) {
  return function () {
    const gen = generatorFn.apply(this, arguments)
    return new Promise((resolve, reject) => {
      function step(key, arg) {
        try {
          const { value, done } = gen[key](arg)
          if (done) return resolve(value)
          Promise.resolve(value).then(
            val => step('next', val),
            err => step('throw', err)
          )
        } catch (e) { reject(e) }
      }
      step('next')
    })
  }
}
```

16. 大厂面试真题

Q: ['1','2','3'].map(parseInt) 的结果是什么?

答案: [1, NaN, NaN]。parseInt 接收两个参数 (值, 基数)。parseInt('1',0) → 1, parseInt('2',1) → NaN (1 进制最大小于 2), parseInt('3',2) → NaN (2 进制最大小于 3)。

Q: 实现 foo(1,2,3).getValue() 返回参数和 (字节)

答案:

```
function foo(...args) {  
  const fn = (...args2) => foo(...args, ...args2)  
  fn.getValue = () => args.reduce((a, b) => a + b, 0)  
  return fn  
}
```

Q: 闭包实现无限累加 add(1)(2)(3)... 最终返回总和

答案:

```
function add(a) {  
  function sum(b) { a += b; return sum }  
  sum.toString = () => a  
  return sum  
}
```

Q: 扫描二维码登录网页的原理 (阿里)

答案: 服务器生成 uid 标识, 二维码对应含 uid 的链接。扫码后客户端登录服务器获取用户信息, 通过长连接通知网页。用户确认授权后, 服务器生成令牌给网页。

Q: React 中 setState 在什么情况下是异步的, 什么情况下是同步的?

答案: React 合成事件和生命周期中是异步的 (批量更新)。setTimeout 和原生 DOM 事件中是同步的。

Q: 页面从 A 打开 B, B 关闭时如何通知 A?

答案: 通信方式: postMessage、localStorage、WebSocket、SharedWorker。正常关闭用 window.onbeforeunload/unload 通知。意外崩溃用心跳监控。

Q: 如何判断 URL 中只包含 qq.com? (腾讯)

答案: 正则为 `/\/\/\w+\.qq\.com[^\.]*$/`。

Q: 实现一个异步求和函数?

答案： 可用 reduce 串行求和或用 Promise.all 分组并行求和优化性能。

Q: Vue2.x 中如何监测数组变化？

答案： 通过函数劫持重写数组变异方法（push/pop/shift/unshift/splice/sort/reverse），指向自定义原型，调用时通知依赖更新。

Q: Proxy 相比于 defineProperty 的优势？

答案： 1. 原生监听数组；2. 监听属性添加和删除；3. 不需深度遍历，getter 时递归代理；4. 13 种拦截方法。

Q: HTML5 为什么只需要写 `<!DOCTYPE HTML>`？

答案： HTML5 不基于 SGML，不需要引用 DTD，只需 DOCTYPE 规范浏览器行为。

Q: 什么是设备像素、CSS 像素、DPR、PPI？

答案： 设备像素是物理像素，不可变。CSS 像素与设备独立像素等价。DPR = 设备像素 / 设备独立像素。PPI 是每英寸像素密度。

Q: 谈一谈 Koa 的洋葱圈模型？

答案： 中间件通过递归的 dispatch 函数依次执行。每次调用 next() 时递归调用下一个中间件，形成洋葱圈结构，在返回时逆序执行后续代码。

整理说明： 本题库由 37 份前端面试 PDF 资料（涵盖 JS、CSS、HTML、浏览器、网络、Vue、React、Webpack、安全、性能优化等方向）提取、去重、归纳而成。内容偏重面试高频考点与核心原理，适合前端工程师系统复习使用。